



Informe Trabajo Práctico Especial
Protocolos de Comunicación
Comisión S

Integrantes:

- Manuel Araujo Feltrin (64090)
- Maria Victoria Alvarez (63165)
- Martín Gallardo Bertuzzi (63510)
- Santiago Sanchez Marostica (64056)

Docentes a cargo:

- Kulesz, Sebastian
- Stupenengo Faus, Hugo Javier
- Axt, Roberto Oscar
- Mizrahi, Thomas

Fecha de Entrega: 11/12/2025

Índice

Introducción.....	3
Descripción de los Protocolos Utilizados.....	3
SOCKS5 Protocol.....	3
S5MP Protocol (SOCKS5 Management Protocol).....	4
Descripción de las Aplicaciones Desarrolladas.....	5
Arquitectura del Servidor.....	5
Componentes Principales.....	5
Problemas Encontrados Durante el Desarrollo.....	5
Limitaciones del Sistema.....	6
Instrucciones de Instalación, Configuración y Uso.....	6
Compilación.....	6
Configuración y Ejecución.....	6
Ejemplos de Prueba.....	7
Pruebas de velocidad.....	8
Pruebas con curl.....	9
Ejemplos de Monitoreo y Administración.....	10
Conexión al Servicio de Management.....	10
Servidor como proxy en Chrome.....	10
Conclusión.....	11
Resultados Obtenidos.....	11
Futuras Mejoras.....	11

Introducción

El presente informe describe el desarrollo del Trabajo Práctico Especial de Protocolos de Comunicación. Para este proyecto se implementa un servidor proxy SOCKS5 completo con capacidades de gestión remota. El objetivo principal es proporcionar un servicio de proxy compatible con RFC 1928 que incluya autenticación de usuarios, resolución DNS asíncrona y un protocolo de gestión personalizado (S5MP) para administración remota. El alcance abarca tanto el servicio de proxy en el puerto 1080 como un servicio de gestión en el puerto 8080, permitiendo monitoreo y administración en tiempo real.

Descripción de los Protocolos Utilizados

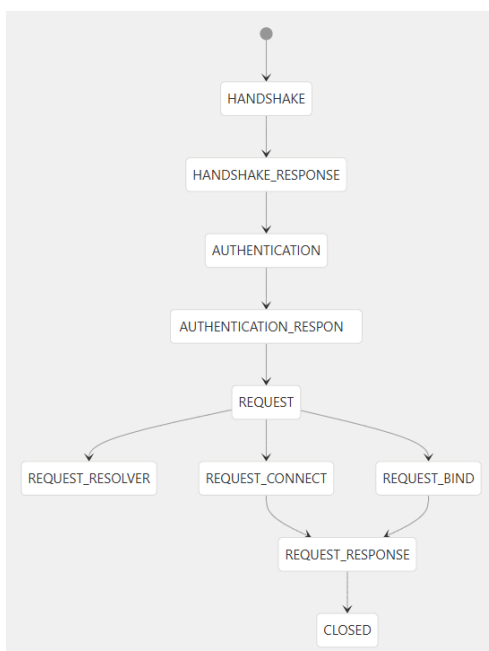
SOCKS5 Protocol

El servidor implementa el protocolo SOCKS5 estándar (RFC 1928) con distintas características técnicas.

Se implementa el flujo de handshake, autenticación y reenvío de conexiones. Por parte del handshake, se recibe la información del cliente a través de buffers y se envía por el mismo medio la respuesta del servidor. Para la autenticación, se realiza utilizando el usuario y la contraseña con el fin de poder asociar cada usuario a su conexión y se cuenta con la posibilidad de realizar algunas funciones sin contar con autenticación. Ya establecida la conexión, se utiliza la state machine además de buffers y el selector para transmitir la información y realizar toda esta operación de lectura y escritura de una manera no bloqueante.

Se dispone de comandos como CONNECT y BIND y distintos tipos de direcciones como serían: IPv4, IPv6 y Domain Name. Asimismo, se tiene resolución DNS que realiza el proxy mediante thread pool asíncrono.

Se puede observar la interacción de la máquina de estados en el siguiente diagrama:



S5MP Protocol (SOCKS5 Management Protocol)

Se implementó un protocolo de texto con delimitador de ‘\n’ con el fin de administrar de manera remota y que facilita la gestión. Se decidió que el protocolo sea basado en texto, ya que facilita la depuración y la validación del intercambio de mensajes, y mejora la legibilidad de la comunicación. Esta decisión implica un mayor overhead en el uso de ancho de banda y en el procesamiento, lo cual puede impactar en el rendimiento en escenarios de alta concurrencia.

Este protocolo corre en el puerto 8080 por defecto y sigue un formato de respuesta similar al de HTTP con códigos 200, 400, 403, 404, 500.

El protocolo S5MP establece un proceso de conexión en dos etapas. Primero, el cliente debe conectarse al puerto correspondiente y enviar el saludo “HELLO S5MP/1.0”. Si el mensaje no coincide exactamente con este formato, el servidor considera la conexión inválida y la cierra. Si el saludo es correcto, responde con “200 S5MP Handshake Successful” y habilita el siguiente paso. A continuación, el cliente puede autenticarse mediante AUTH <user> <password>: si las credenciales son válidas, el servidor devuelve “200 OK” y concede acceso a los comandos disponibles para el rol asociado; si no lo son, responde “401 Unauthorized” y cierra la conexión. Sin embargo, cuando el servidor se ejecuta sin un usuario administrador configurado, la autenticación no es requerida: en ese modo, el protocolo permite utilizar directamente los comandos disponibles para el rol user, sin necesidad de enviar AUTH y sin exigir credenciales. Esto facilita el uso del servidor en entornos de prueba o configuración básica.

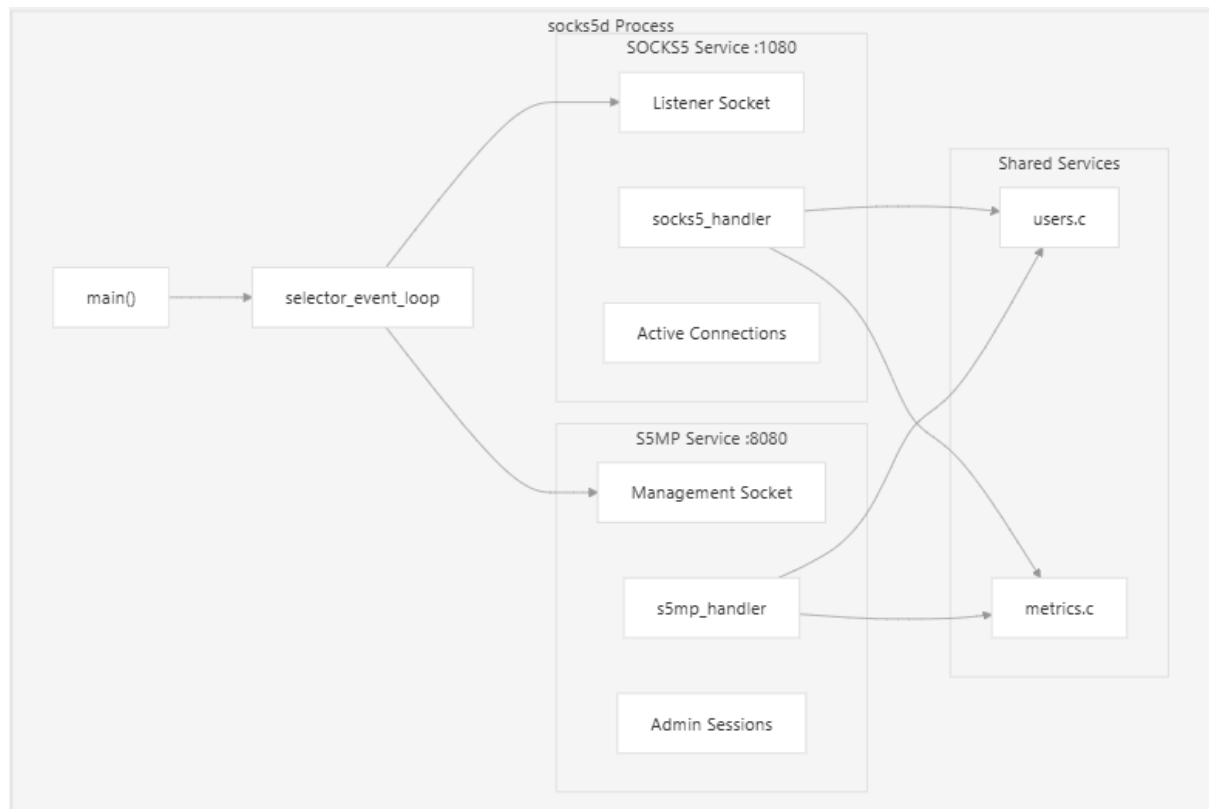
Se cuenta además con distintos comandos como: GET_METRICS, GET_LOGS, USERS, ADD_USER, DELETE_USER, ROLE_SETTER, BUFFER_NEWSIZE, QUIT. Estos facilitan la implementación del protocolo socks5. A continuación, se presenta un ejemplo de uso del protocolo:

```
maraujofeltrin@PCManu:~/Protos$ nc 127.0.0.1 8080
HELLO S5MP/1.0
200 S5MP Handshake Successful
AUTH admin pass123
200 OK
GET_METRICS
200 OK
active_connections 0
total_connections 0
total_bytes_transferred 0
```

Descripción de las Aplicaciones Desarrolladas

Arquitectura del Servidor

El servidor socks5 implementa una arquitectura de doble servicio en un único proceso:



Componentes Principales

Se cuenta con un event selector loop que permite manejar el sistema de I/O no bloqueante. Además, se tiene una máquina de estados que es utilizada tanto por socks5 como por el protocolo S5MP. También, se cuenta con un sistema de buffers dinámicos con tamaño configurable y un sistema de métricas con estadísticas de conexiones y bytes transferidos.

Problemas Encontrados Durante el Desarrollo

Se enfrentaron distintos problemas en la creación del proyecto. Uno de estos siendo que se realizaban cuatro syscalls (selector → read → selector → write) por cada byte, haciendo que se pierda eficiencia. Esto se solucionó reduciendo a tres syscalls (selector → read → write) cada 4K bytes. Otro fue la prueba de la conexión IPv6, ya que se intentó utilizar esta conexión en pamparo mientras que este no la soportaba.

Además, se tuvo problemas a la hora de implementar autenticación ya que inicialmente se intentó tener un usuario default creado a la hora de inicializar el servidor. Esto fue corregido debido a que en la pre-entrega se indicó que, a la hora de utilizar el proxy sin

autenticación pero con el usuario definido se colgaba. Se decidió implementar que una vez iniciado el servidor cuando se cree el primer usuario, se le adjudique el rol de administrador.

Limitaciones del Sistema

Una limitación es un máximo de 256 usuarios autenticados concurrentes y un buffer de 4096 bytes por conexión. Además, se limitó a un buffer circular de 1024 entradas para los logs y se limitó a 1024 file descriptors concurrentes siendo también este número el máximo de conexiones activas simultáneas.

Por parte de seguridad, se limitó a un sistema básico de autenticación contando con username y password que cumple con el RFC 1929. Además, no se utilizó ninguna encriptación a la hora de realizar el guardado de contraseña.

Instrucciones de Instalación, Configuración y Uso

Compilación

```
make all      # Compila todo (server, client, utils, tests)
make server   # Compila solo el servidor
make client   # Compila solo el cliente
make clean    # Limpia binarios y objetos
```

Configuración y Ejecución

El servidor acepta los siguientes parámetros:

```
# Ejecución con defaults
./build/bin/socks5d

# Es obligatorio especificar usuario para acceder a métricas
./build/bin/socks5d -u admin:pass123

# Configuración personalizada
./build/bin/socks5d -l 0.0.0.0 -p 1080 -L 127.0.0.1 -P 8080 -u user123:pass123

# Múltiples usuarios (solo el primer usuario se crea con rol de admin)
./build/bin/socks5d -u alice:alicepwd -u bob:bobpwd

# Para que el servidor acepte IPv6
./build/bin/socks5d -l :: -p 1080
```

Debe especificarse manualmente un usuario administrador mediante la opción -u (por ejemplo: admin:pass123) al levantar el servidor para acceder a toda la sección de monitoreo y administración que requieran permisos de administrador.

Ejemplos de Prueba

Se realizaron múltiples pruebas con el fin de evaluar el correcto funcionamiento del servidor socks5. Para esto se decidió hacer distintas pruebas, incluyendo pruebas de estrés.

En un comienzo, se realizaron pruebas más simples de funcionamiento.

Ejecución de comando de listado de usuarios:

```
martin19@Martin:~/TPE-Protos/TPE-Protos$ ./build/bin/socks5_client -u admin:pass123 -U
User 1: admin, Role: admin
```

Ejecución de comando de versión:

```
martin19@Martin:~/TPE-Protos/TPE-Protos$ ./build/bin/socks5_client -v
SOCKS5 Management Client v1.0
```

Se establece sesión y añade usuario:

```
martin19@Martin:~/TPE-Protos/TPE-Protos$ ./build/bin/socks5_client -u admin:pass123 -a user:123
User added successfully.
```

Ejecución de comando de cambio de rol:

```
martin19@Martin:~/TPE-Protos/TPE-Protos$ ./build/bin/socks5_client -u admin:pass123 -r user:admin
Role updated successfully.
```

Ejecución de comando de borrado de un usuario:

```
martin19@Martin:~/TPE-Protos/TPE-Protos$ ./build/bin/socks5_client -u admin:pass123 -d user
User deleted successfully.
```

Ejecución de comando de cambio de tamaño del buffer:

```
martin19@Martin:~/TPE-Protos/TPE-Protos$ ./build/bin/socks5_client -u admin:pass123 -b 2048
Buffer size updated successfully.
```

Luego, se comenzaron las pruebas de estrés manteniendo 500 conexiones activas realizando una descarga limitada a 1K de un archivo de 100MB. Esto se hizo para corroborar que el servidor soporta 500 conexiones socks5 a la vez.

```
martin19@Martin:~/TPE-Protos/TPE-Protos$ for i in $(seq 1 500); do curl -x socks5h://admin:pass123@127.0.0.1:1080 --limit-rate 1k -o /dev/null -s http://ipv4.download.thinkbroadband.com/100MB.zip & done ;
```

```

martin19@Martin:~/TPE-Protos/TPE-Protos$ ./build/bin/socks5_client -u admin:pass123 -m
Total Connections: 500
Active Connections: 500
Total Bytes Transferred: 36112044
martin19@Martin:~/TPE-Protos/TPE-Protos$ ./build/bin/socks5_client -u admin:pass123 -m
Total Connections: 500
Active Connections: 500
Total Bytes Transferred: 338706396
martin19@Martin:~/TPE-Protos/TPE-Protos$ ./build/bin/socks5_client -u admin:pass123 -m
Total Connections: 500
Active Connections: 500
Total Bytes Transferred: 569432164

```

Pruebas de velocidad

Se realizó una prueba de velocidad de descarga con defaults, siendo estos un buffer de 4KB.

```

maraujofeltrin@PCManu:~/Protos$ curl -x socks5h://admin:pass123@127.0.0.1:1080 -o /dev/null -w "time_total: %{time_total}s\nspeed_download:
%{speed_download} bytes/s\n" http://ipv4.download.thinkbroadband.com/10MB.zip
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
   Dload  Upload  Total   Spent    Left     Speed
100 10.0M  100 10.0M    0     0 1861k      0  0:00:05  0:00:05 --:--:-- 2187k
time_total: 5.500590s
speed_download: 1906297 bytes/s

```

Posteriormente, se realizó una prueba de velocidad de descarga sin pasar por el proxy.

```

maraujofeltrin@PCManu:~/Protos$ curl -o /dev/null -w "time_total: %{time_total}s\nspeed_download: %{speed_download} bytes/s\n" http://ipv4.d
ownload.thinkbroadband.com/10MB.zip
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
   Dload  Upload  Total   Spent    Left     Speed
100 10.0M  100 10.0M    0     0 2180k      0  0:00:04  0:00:04 --:--:-- 2251k
time_total: 4.697071s
speed_download: 2232403 bytes/s

```

Se puede observar que en la descarga con proxy se tienen 1,816,824 bytes/s (≈ 1.73 MB/s) en 5.77s, mientras que con el proxy se tuvieron 1,755,734 bytes/s (≈ 1.67 MB/s) en 5.97s. Dando así un overhead del proxy $\sim 3.4\%$ que resulta prácticamente despreciable.

Por otra parte, se realizaron pruebas de carga. Para esto se necesitó crear un servidor local en el puerto 8082 con el fin de obtener resultados sin interferencias externas. Además de esto se creó un archivo de 10MB para utilizar como recurso durante las pruebas. Se obtuvo con las pruebas de carga con proxy:

#Comando utilizado para la prueba de carga

```

echo "==== TEST UPLOAD 10MB VIA PROXY ====" && timeout 10 curl -x
socks5h://admin:pass123@127.0.0.1:1080 -T test_10MB.bin -o /dev/null -w
"HTTP:%{http_code} Speed:%{speed_upload} bytes/s | Time:%{time_total}s\n"
http://127.0.0.1:8082/upload

```



```

=== TEST UPLOAD 10MB SIN PROXY ===
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload  Total  Spent    Left     Speed
100 10.0M    0   18 100 10.0M    17 9973k  0:00:01  0:00:01  --:--:-- 9980k
HTTP:200 Speed:10213011 bytes/s | Time:1.026706s

```

Mientras que para las pruebas de carga sin proxy:

#Comando utilizado para la carga sin pasar por el servidor proxy

```

echo "=== TEST UPLOAD 10MB SIN PROXY ===" && head -c 10M /dev/urandom >
test_10MB.bin && timeout 10 curl -T test_10MB.bin -o /dev/null -w
"HTTP:%{http_code} Speed:%{speed_upload} bytes/s | Time:%{time_total}s\n"
http://127.0.0.1:8082/upload

```

```

=== TEST UPLOAD 10MB SIN PROXY ===
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload  Total  Spent    Left     Speed
100 10.0M    0   18 100 10.0M    17 9973k  0:00:01  0:00:01  --:--:-- 9980k
HTTP:200 Speed:10213011 bytes/s | Time:1.026706s

```

En base a estas pruebas de carga, se observó que sin el proxy se tiene una velocidad de 10,213,011 bytes/s con un tiempo total de 1.03s, mientras que con el proxy se tiene una velocidad de 9,867,325 bytes/s con un tiempo total de 1.06s. Esto da como resultado un overhead del proxy de ~3.4%.

Pruebas con curl

Si el servidor se inicializa sin ningún usuario, el curl puede realizarse sin autenticación. Si el servidor se inicializa con usuarios, es necesario validar las credenciales.

Validación con curl sin usuarios

```

# Proxy básico
curl --proxy socks5h://127.0.0.1:1080 https://www.google.com
# Con modo verboso
curl -v --socks5 127.0.0.1:1080 http://www.google.com

# Con IPv6
curl -6 --proxy socks5h://[::1]:1080 https://www.google.com

```

Validación con curl con usuarios

```
# curl con autenticación  
curl -v --socks5 admin:pass123@127.0.0.1:1080 http://www.google.com
```

Ejemplos de Monitoreo y Administración

Conexión al Servicio de Management

```
./build/bin/socks5_client -u admin:pass123 [opciones]
```

Operaciones Comunes

```
# Obtener métricas  
./build/bin/socks5_client -u admin:pass123 -m  
  
# Listar usuarios  
./build/bin/socks5_client -u admin:pass123 -U  
  
# Agregar usuario  
./build/bin/socks5_client -u admin:pass123 -a newuser:newpass  
  
# Eliminar usuario  
./build/bin/socks5_client -u admin:pass123 -d newuser  
  
# Cambiar rol  
./build/bin/socks5_client -u admin:pass123 -r newuser:admin  
  
# Cambiar tamaño de buffer  
./build/bin/socks5_client -u admin:pass123 -b 2048  
  
# Obtener logs  
./build/bin/socks5_client -u admin:pass123 -l
```

Servidor como proxy en Chrome

Para utilizar el servidor como proxy en Google Chrome es necesario ejecutarlo sin autenticación de usuarios, ya que el navegador no implementa el mecanismo de autenticación definido por SOCKS5. Por este motivo, basta con iniciar el servidor con su configuración por defecto.

Luego, Chrome debe iniciarse especificando explícitamente el proxy a utilizar. Para ello se emplea el siguiente comando:

#Correr Chrome con el siguiente flag
chrome.exe --proxy-server="socks5://127.0.0.1:1080"

Es importante asegurarse de que todas las ventanas de Chrome estén cerradas previamente, ya que el navegador reutiliza procesos en segundo plano y podría ignorar la configuración del proxy.

Una vez ejecutado, Chrome utilizará el servidor SOCKS5 para todo su tráfico. Para verificar su funcionamiento se realizaron pruebas de navegación, incluyendo búsquedas en Google, reproducción de videos en YouTube en alta resolución y la apertura de 25 pestañas simultáneas para observar el comportamiento de las conexiones. Todas las pruebas se completaron correctamente, confirmando el uso efectivo del proxy.

Conclusión

Resultados Obtenidos

El servidor utiliza una arquitectura de doble protocolo que permite la separación de tráfico de proxy y gestión, mejorando la seguridad y facilitando la administración. La implementación sigue de cerca el estándar RFC 1928 para SOCKS5 mientras que S5MP proporciona una interfaz de administración simple pero efectiva basada en texto.

Se implementó exitosamente un servidor proxy SOCKS5 completo con capacidades de gestión remota. La arquitectura monohilo basada en eventos demuestra buen rendimiento para cargas moderadas, y el protocolo S5MP proporciona una interfaz administrativa funcional.

Futuras Mejoras

Unas posibles mejoras a futuro podría ser el cambio de select por epoll, ya que permitiría una mayor cantidad de conexiones simultáneas al aumentar la cantidad de file descriptors disponibles. Otra mejora podría ser una mejor autenticación, sea esto implementar más métodos indicados en el RFC 1928.

Una considerable mejora podría ser la implementación de una blacklist, lo que permitiría la limitación de IP's o dominios. Esto permitiría filtrar el acceso a ciertos sitios como un proxy. Por último, una mejora sería el poder ofrecer el soporte UDP_ASSOCIATE permitiendo así el reenvío de tráfico UDP a través del proxy.